

Relatório de Atividade Prática - Revisão para a Prova 2

Disciplina: Integração e Entrega Contínua (IEC)

Tema: Revisão Geral – Testes, Cobertura, Segurança e Badges

Estudante: Alisson Gritti

Curso: Superior de Tecnologia em Desenvolvimento de Software Multiplataforma (DSM)

PARTE I - QUESTÕES DE MÚLTIPLA ESCOLHA

- **Questão 1:** c) Uma parcela significativa do código pode não estar sendo validada pelos testes.
- **Questão 2:** b) detectar problemas antes da implantação.
- **Questão 3:** c) identificar vulnerabilidades em dependências.
- **Questão 4:** b) a cobertura dos testes precisa ser ampliada..
- **Questão 5:** c) Fornecem indicadores visuais sobre a saúde do projeto.

PARTE II - QUESTÕES DISCURSIVAS

Questão 6

- **Teste Automatizado:** É a execução programada de scripts de código para validar se uma funcionalidade se comporta como esperado, dispensando validações manuais humanas. *Exemplo no INPE:* Um teste que envia dados de foco de calor e checka se o sistema retorna a classificação correta.
- **Cobertura de Testes (Coverage):** É a métrica percentual que indica quantas linhas, funções ou ramos (branches) do código fonte real foram de fato executados durante a rodada de testes. *Exemplo no INPE:* Identificar se a função que dispara alertas de enchentes por SMS foi testada ou ignorada.
- **Vulnerabilidade de Software:** É uma brecha, falha ou fragilidade de segurança real presente no código ou em dependências de terceiros que pode ser explorada por agentes mal-intencionados para invadir o servidor ou roubar dados. *Exemplo no INPE:* Uma brecha na biblioteca de envio de e-mails que permita a um hacker disparar alertas falsos de evacuação para a população.

Questão 7

Essa situação representa um risco crítico porque **60% do projeto está completamente sem testes**. Embora existam 300 testes funcionais, eles estão concentrados em uma parte isolada do software. Se uma regra de negócio complexa do INPE (como o cálculo de

dispersão de focos de incêndio) sofrer alterações nessa parcela não coberta, o sistema poderá falhar gravemente em produção e a equipe só descobrirá quando os alarmes pararem de tocar para a população.

Questão 8

Recomendação: Prioridade absoluta na escrita de novos testes unitários focados na camada de **Utils**, paralisando novos deploys até que este índice suba.

Justificativa: Enquanto **Controllers** e **Services** possuem uma segurança excelente (acima de 90%), a camada **Utils** possui apenas 18%. Arquivos utilitários geralmente guardam funções globais compartilhadas por todo o projeto (como cálculos matemáticos climáticos ou formatadores de data). Se um arquivo central desses quebrar, causará um efeito dominó que derrubará o sistema inteiro, tornando-se o ponto mais arriscado do projeto atual.

Questão 9

A alta cobertura garante que o robô passou pelas linhas de código, mas **não garante que o software esteja livre de erros**. Isso ocorre porque:

1. **Dados inesperados:** Os testes podem usar dados controlados e perfeitos, mas o usuário final em produção pode inserir interações e cenários imprevistos que quebram a lógica.
2. **Asserções fracas:** O teste pode passar pelas linhas de código (gerando cobertura), mas não conter validações (**expect**) rígidas o suficiente para checar se o valor retornado está semanticamente correto.
3. **Erros de especificação:** O código e o teste podem estar tecnicamente perfeitos, mas a regra de negócio implementada foi mal interpretada ou especificada erroneamente desde o início.

Questão 10

- **O que faz:** Aciona a suíte de testes automatizados do projeto através do atalho configurado na propriedade "**test**" do arquivo **package.json** (geralmente executando ferramentas como o Jest).
- **Quando é executado:** É executado de forma automática no pipeline de Integração Contínua (CI) sempre que um desenvolvedor realiza um **push** ou abre um **Pull Request** nas branches protegidas.
- **Importância no INPE:** Atua como um "porteiro" automatizado. Garante de forma ágil e segura que nenhuma nova funcionalidade ou correção enviada pela equipe quebre o código que já estava funcionando, blindando o aplicativo de monitoramento climático contra falhas operacionais involuntárias.

Questão 11

Não concordo. Prosseguir com o deploy contendo duas vulnerabilidades de nível **High** é colocar a segurança da produção e a confiabilidade do INPE em risco direto. As ferramentas de CI e auditoria existem exatamente para impedir que essas brechas cheguem à produção. A equipe deveria travar o pipeline, executar o comando `npm audit fix` para atualizar os pacotes e, caso quebrasse o sistema, buscar uma biblioteca alternativa segura antes de liberar o deploy.

Questão 12

- **Conceito:** Refere-se a falhas ou brechas de segurança conhecidas que existem dentro de bibliotecas ou pacotes criados por terceiros e que nós importamos para dentro do nosso projeto (gerenciadas pelo arquivo `package.json`).
- **Exemplo no INPE:** O aplicativo utiliza uma dependência externa para gerar os mapas e gráficos de calor das queimadas. Se descobrirem uma vulnerabilidade do tipo *Remote Code Execution* nessa biblioteca de mapas, um invasor externo poderia se aproveitar dessa brecha para executar scripts maliciosos, roubar os dados meteorológicos ou derrubar o servidor do INPE.

Questão 13

Não garante. Conforme estabelecido na engenharia de software, a cobertura de 97% prova que os testes passam por quase todas as linhas de código existentes, mas ela é incapaz de prever falhas de hardware, erros de lógica de negócio mal interpretados ou brechas de segurança em tempo de execução criadas por interações do usuário que não foram desenhadas nos cenários dos testes unitários originais.

Questão 14

Os relatórios auxiliam o gerente atuando como uma métrica quantitativa clara para governança e controle de qualidade. Se o relatório indicar uma queda brusca de cobertura ou um índice abaixo da linha de corte acordada pela governança do projeto, o gerente tem o embasamento técnico necessário para travar o deploy e determinar o retorno da tarefa para a equipe de desenvolvimento, mitigando riscos de prejuízo ou falhas graves antes da entrega.

Questão 15

Recomendação: Barrar o deploy imediatamente para correção das dependências.

Justificativa: Apesar dos excelentes índices de qualidade técnica (100% dos testes passando e 86% de cobertura), a presença de **5 vulnerabilidades do tipo High** invalida a segurança do produto. Fazer o deploy desse cenário no INPE expõe um sistema de utilidade pública a ataques cibernéticos iminentes, podendo resultar no sequestro do servidor ou na manipulação dos dados de alertas de queimadas emitidos para a população.

Questão 16

Os testes automatizados formam a base de sustentação da Integração Contínua (CI). A essência da CI é permitir que desenvolvedores integrem código em um repositório compartilhado várias vezes ao dia. Essa integração frequente só é segura porque, a cada envio, os **testes automatizados rodam sozinhos no pipeline**, validando se as novas alterações introduziram regressões ou quebraram funcionalidades existentes de forma imediata.

Questão 17

Essa funcionalidade lida diretamente com a segurança civil e ambiental. Se o fluxo de denúncias falhar por conta de um bug pós-deploy, focos de incêndio reais podem deixar de ser computados e combatidos pelas autoridades. Uma estratégia rigorosa de testes automatizados valida que o aplicativo continuará recebendo as denúncias, tratando os anexos e integrando os dados com precisão, mesmo sob alta carga de acessos.

Questão 18

- **Build:** Mostra se o código compila com sucesso, se as dependências foram instaladas e se o ambiente básico de execução foi estruturado sem erros fatais.
- **Tests:** Exibe de forma rápida a quantidade de testes unitários ou de integração que passaram com sucesso versus os que falharam na última execução.
- **Coverage:** Demonstra o percentual geral de linhas de código que foram cobertas e validadas pela suíte de testes executada.
- **Vulnerabilities:** Indica se o projeto possui dependências com brechas conhecidas de segurança e o nível de gravidade atual delas (Low, Medium, High, Critical).

Questão 19

A afirmação é **tecnicamente incorreta e perigosa**. A cobertura de testes avalia apenas a *quantidade* de linhas executadas, mas não a *qualidade* ou o significado do código. A revisão de código humana (Code Review/Pull Request) é indispensável para avaliar legibilidade, boas práticas de arquitetura, padrões de design do projeto, possíveis gargalos de performance e se a solução desenhada faz sentido estratégico para o negócio.

Questão 20

Como responsável pela aprovação do deploy do INPE, eu analisaria:

1. **Status do Build/Pipeline (Sucesso/Falha):** Garante que o software compila e roda no ambiente de destino.
2. **Resultado dos Testes Automatizados (100% de Sucesso):** Certifica que nenhuma regra de negócio crítica existente foi quebrada pelas novas alterações.
3. **Métrica de Cobertura de Código (Mínimo de 80%):** Garante que as novas lógicas não estão indo às cegas e foram minimamente validadas.
4. **Relatório do npm audit (Zero vulnerabilidades High/Critical):** Garante que o servidor de produção e os dados dos alertas climáticos emitidos para a população não estarão expostos a ataques de hackers.

